

 Design status: all sections drafted · v0.1

SYSTEM DESIGN DOCUMENT · TAKE-HOME

Status Page Platform System Design

A multi-tenant status-page service in the spirit of incident.io, Atlassian Statuspage, and Rootly. This document covers the high-level design end to end: requirements, architecture, data model, APIs, deployment, and the trade-offs behind each choice.

Scope: High-level design (HLD) Reader: Engineering reviewer
Low-level design: separate follow-up Diagrams: placeholders, built in Eraser

CONTENTS

01	Overview & goals	context
02	Requirements & scale	scope
03	System context	diagram
04	High-level design	core
05	Tech stack & rationale	decisions
06	Data model & tables	schema

07	State diagrams	lifecycle
08	API surface	contract
09	Deployment & monitoring	ops
10	Trade-offs & future work	honest

01

Overview & goals

What we're building and the one idea that shapes everything.

A **status page** is the public board a company shows its customers during an outage or maintenance: which parts of the product are healthy, what's currently broken, and what the team is doing about it. We're building this as a **multi-tenant SaaS**, where many customer organisations (tenants) each get their own branded page on their own domain, like `status.acme.com`.

The product sits downstream of monitoring. **Signals already exist** in the customer's world (Datadog, PagerDuty, Prometheus, and so on). We don't generate them. We consume them, and add a human-friendly communication layer on top.

KEY INSIGHT: THE WHOLE DESIGN FALLS OUT OF THIS

Reads and writes are wildly asymmetric. The public page is *read* by potentially millions of people during an outage, and every reader sees the *same* page. It is *written* only a handful of times per incident. So we split the system into a **read plane** (a pre-computed page served from a CDN, cheap and outage-proof) and a **control plane** (where the rare, careful writes happen). Almost every decision in this document traces back to that split.

Goals

Stay up when the customer is down. The page is most important precisely when things are broken, so it must not share fate with either the customer's product or our own control plane.

Absorb traffic spikes. A big tenant's outage can drive a sudden flood of readers. The read path must shrug that off.

Communicate reliably. Subscribers get notified on every meaningful update, never twice and never missed.

Fit the responder's workflow. Updates should flow from the incident itself, automatically or with one click, not as separate copy-paste busywork.

Non-goals

We do **not** build monitoring, metrics storage, alerting rules, or anomaly detection. Those produce the signals; we consume them.

We do **not** build importers for migrating off other status-page vendors (kept as a clean extension point, not designed here).

02

Requirements & scale

Functional, non-functional, and the numbers that justify the architecture.

Functional requirements

A tenant-scoped **public status page**, served on a custom domain with custom branding.

Components and component groups, each carrying a health state: **operational**

degraded **partial outage** **major outage** **maintenance**

Incidents with a lifecycle (**investigating** → **identified** → **monitoring** → **resolved**), carrying threaded **updates**, each affecting one or more components.

Auto-detected incidents from signals arrive as **proposed** and wait for human approval. Responders can also create and resolve incidents manually.

Scheduled maintenance with its own lifecycle (**scheduled** → **in_progress** → **completed**).

Uptime history and a public **past-incidents** timeline.

Subscriptions & notifications across email, RSS, and webhook (SMS / Slack as fast-follow).

An **admin / responder dashboard** to manage everything above and approve proposals.

Non-functional requirements

Outage-independent reads. The published page is static and edge-served, independent of the control-plane database. If the database is down, the last published page still serves.

Read/write asymmetry. Reads dominate by 4 to 5 orders of magnitude and are highly cacheable. Writes are rare but expensive (they fan out).

Idempotent fan-out. Notifications are never double-sent and never silently dropped.

Tenant isolation. One tenant's traffic or incident never degrades another's.

◯ IDEMPOTENT, IN ONE LINE

Doing the same operation twice has the same effect as doing it once. If a notification job runs again after a crash, the subscriber still gets exactly one message.

Back-of-envelope scale

DIMENSION	ESTIMATE	CONSEQUENCE
Tenants	~10K, each with ~10 to 50 components	Tenancy is first-class on every table.
Subscribers	avg ~1K/tenant; large tenants 100K to 1M+	Fan-out is the real write-side cost.
Reads	near-zero baseline, then 50K to 100K+ RPS spikes on one page	Static JSON at a CDN; DB never touched on reads.

DIMENSION	ESTIMATE	CONSEQUENCE
Writes	a handful per minute, globally	Trivial to serve; the cost is the fan-out it triggers.

THE ONE SENTENCE TO REMEMBER

Reads are massive, identical, cacheable, and must outlive the system itself; writes are rare but trigger expensive, must-not-fail fan-out. Two planes, connected by a single publish step.

03

System context

Who talks to the system, and what crosses its boundary.

Three kinds of actor interact with the platform. **Signal sources** push alerts in. **Tenant admins / responders** manage incidents and approve proposed updates. **End users** read the public page, and a subset become **subscribers** who receive notifications. Outbound, the system talks to third-party delivery providers (email, SMS) and a CDN edge.



DIAGRAM PLACEHOLDER · 01

System context diagram

Actors (signal sources, tenant admin, end users, subscribers) around the platform boundary, showing the control plane publishing to the read plane and notifying subscribers. *Insert exported Eraser image or paste the diagram-as-code script here.*

WHY A CONTEXT DIAGRAM FIRST

It fixes the boundary before we open the box. Everything inside is ours to design; everything outside is a contract we either consume (signals) or fulfil (the public page, notifications).

04

High-level design

The two planes, every component's job, and the paths data takes through them.

4.1 The two planes

The system divides cleanly into a **read plane** and a **control plane**, joined by a one-way **publish** step. The control plane holds the truth and does the writing; the read plane holds a pre-computed, edge-served copy that the public sees.

THE NEWSPAPER PICTURE

The newsroom (control plane) prints an edition *once*. Trucks carry identical copies to newsstands worldwide (the CDN). Readers buy from the nearest newsstand and never call the newsroom. If the newsroom burns down overnight, every copy already on a newsstand is still readable. That last property is how the page survives our own outage.



DIAGRAM PLACEHOLDER · 02

Architecture diagram (read plane / control plane / workers)

The full component architecture with AWS / tech logos. The approved Eraser script is included below. Paste it into an Eraser diagram (code mode) for free.

► Eraser diagram-as-code, paste into Eraser (no AI credit needed)

4.2 Components & responsibilities

Each box does one job. Plane tags: **read** **control** **async**

READ **CDN**

Caches and serves the published page from edge locations near readers. Absorbs the thundering herd; the origin is only hit on a cache miss.

READ **Snapshot store (object storage)**

Holds the pre-computed status JSON, the "printed editions." It is the CDN's origin: cheap, durable, and independent of the database.

READ **Public page (static frontend)**

The visitor-facing UI. Static files that fetch the status JSON through the CDN. Never talks to the control plane live.

CONTROL **Backend API**

The brain. Ingests signals, owns incident logic, runs the approval workflow, writes to the database, triggers publishing, and enqueues notifications.

CONTROL **Admin dashboard (frontend)**

Where responders create, update, and resolve incidents and approve proposed updates. Talks to the Backend API.

CONTROL PostgreSQL

The source of truth: tenants, components, incidents, updates, subscribers, templates, audit log.

CONTROL Redis

Fast-path memory: deduplication keys for incoming signals, rate-limit counters, and short-lived caching.

ASYNC Publisher

Renders the current state into a status JSON snapshot, writes it to the snapshot store, and invalidates the CDN. The single gate where state becomes public.

ASYNC Queue + notification workers

The queue holds fan-out tasks; workers deliver them per channel (email, SMS, webhook) with retries and idempotency, outside the web request.

CONTROL AI service (optional)

Drafts customer-facing update wording and post-incident summaries. Always human-approved; never in the critical path.

4.3 Write path (incident → public)

- ¹ A signal arrives at the ingestion endpoint, *or* a responder opens an incident in the dashboard.
- ² The API checks Redis for the signal's dedup key and drops repeats; survivors are written to Postgres.
- ³ An auto-detected incident is saved as `proposed` ; a manual one is authored directly.
- ⁴ The update is finalised through one of three modes (next section), then the **Publisher** renders a fresh snapshot and pushes it to the CDN.

- 5 The same publish step enqueues notification tasks for affected subscribers.

4.4 Read path (visitor → page)

- 1 A visitor opens the tenant's status page (static files, served from the CDN).
- 2 The page fetches the status JSON, also through the CDN, backed by the snapshot store.
- 3 No database is touched. The page renders from cached static content, so it stays fast under load and survives a control-plane outage.



DIAGRAM PLACEHOLDER · 03

Write path & read path (two flow diagrams)

One flowchart for the write path (incident → publish → notify), one for the read path (visitor → CDN → snapshot). *Diagram-as-code scripts to follow.*

4.5 Signal ingestion

We mirror how incident.io and Rootly take signals in: **HTTP endpoints, never database access**. Each tenant configures one or more **alert sources**; each source gets a unique webhook URL and an auth token. Native integrations cover common tools; a generic HTTP source covers anything else that can send a webhook.

Dedup key. Every alert carries a fingerprint. We remember recent fingerprints in Redis, so a flapping alert firing every 30 seconds doesn't create 100 incidents.

Rate limit. Each source is capped (e.g. ~120 events/min) so a misbehaving source can't overwhelm ingestion.

Adapter layer. Sources translate into one normalised internal signal format, so swapping or adding a source never touches the core.

WHY ENDPOINTS, NEVER THE DATABASE

A waiter takes your order; you don't walk into the kitchen. Endpoints give security (no peeking at other tenants' data), freedom (we can change our schema without breaking customers), and a small blast radius (one bad caller can't take down shared infra). DB access would surrender all three.

4.6 Incident lifecycle & the three update modes

The internal incident is the **source of truth**; the public status page is a cleaned-up **projection** of it: the same truth, made safe for customers. An update reaches the page in one of three modes, all funnelling through the same publish gate:

Automatic. A simple rule posts the update ("if SEV1, post Investigating"). No human in the loop.

Approval template. The system drafts the message from a pre-approved template; a human reviews and clicks approve. This is the `proposed` state made concrete.

Manual. A responder writes and posts the update from scratch.

ONE GATE, ON PURPOSE

All three modes pass through a single publish step. That gives us exactly one place to enforce safety checks, write the audit trail, and fire notifications, instead of three slightly-different code paths that drift apart over time.

4.7 Notification fan-out

One published update may need to reach up to a million subscribers across channels. That can't happen inside a web request, so the publish step drops tasks into a **queue**, and background **workers** deliver them, retrying failures and keying on an idempotency token so nobody gets a duplicate.

THE POST-OFFICE PICTURE

You drop one announcement at the counter (publish). Behind the scenes, carriers (workers) copy and deliver it to every address (subscribers), some by

email route and some by SMS. A failed delivery is retried by that carrier and never holds up the counter.



DIAGRAM PLACEHOLDER · 04

Notification fan-out

Publish → queue → email / SMS / webhook workers → providers → subscribers. *Diagram-as-code script to follow.*

TRADE-OFF: KEEP THE QUEUE AN EXTENSION POINT

A full queue + worker fleet is the most complex part to operate. So we design the *seam* (a `NotificationDispatcher` abstraction) from day one but start with the simplest implementation. The real queue is the first scale-up lever, and swapping it in requires no rewrite of incident logic.

4.8 AI assist (optional, at the edges)

AI shows up in two non-critical spots, both opt-in and both human-approved:

Update drafting. Turn the messy internal incident notes into a clean customer-facing message, which a responder then approves. This plugs straight into the approval queue we already have.

Postmortem assist. After resolution, draft a retrospective from the incident timeline, reducing toil and helping deflect repeat support tickets.

DELIBERATELY MINOR

AI never decides what goes public on its own and never sits in the read path. It's an assistant on the side of the workflow, not a component the system depends on.

05

Tech stack & rationale

The chosen stack, with a short reason for each. Two lines, not a debate.

LAYER	CHOICE	WHY
Backend API	Java + Spring Boot	Strongly OOP and modular, which keeps the domain logic clean and testable as it grows.
Frontend	React / Next.js	The public page renders to static files (edge-served); the admin dashboard is a React app talking to the API.
Database	PostgreSQL	The data is relational (tenants, components, incidents, subscribers), so a relational database is the natural fit.
Fast-path store	Redis	In-memory speed with auto-expiry for dedup keys, rate limiting, and short-lived caches.
Queue	SQS (or similar)	Decouples the rare write from the expensive fan-out; durable and simple to operate.
Snapshot store	S3	Cheap, durable object storage for published page snapshots; doubles as the CDN origin.
CDN	CloudFront	Edge caching for the read path; also terminates TLS for tenant custom domains.
Email / SMS	Third-party providers	Deliverability is a hard problem best bought, not built. Wrapped behind a provider interface so channels are swappable.
Runtime	Docker containers	Portable; runs on a simple platform early and on managed orchestration at scale.

TLS, QUICKLY

TLS is what turns `http` into `https`, the padlock. It seals data in transit so only the right site can read it, and proves the visitor is on the real page. Each tenant's custom domain needs its own certificate, issued automatically.

CLOUD: AWS BY DEFAULT, NOT A LOCK-IN

We use AWS because it has the most mature, widely-used managed services and most teams already know it. The design isn't tied to it. Every piece maps to a GCP equivalent (Cloud Storage and Cloud CDN, Cloud SQL, Pub/Sub, Cloud Run or GKE), and since email runs on a third-party provider, that part is cloud-agnostic. So we could switch if needed.

EXTENSION POINTS, DESIGNED-IN BUT NOT BUILT

The system is built to be deployed and observed with infrastructure-as-code and metrics dashboards (e.g. Terraform, Prometheus + Grafana) as it grows. These are deliberately left as clear extension points rather than day-one work.

06

Data model & tables

The core entities and how they relate. Every table is tenant-scoped.

The schema centres on the **incident** (the source of truth) and the **component** (what has a health state). Updates thread off incidents; a join table records which components an incident affects. Subscribers and notifications live on the communication side. Everything carries a `tenant_id` so tenants stay isolated.



DIAGRAM PLACEHOLDER · 05

Entity-relationship diagram (ERD)

Crow's-foot ERD of the tables below. *Eraser ERD diagram-as-code script to follow, pasteable.*

Core tables

TABLE	KEY COLUMNS	PURPOSE
tenants	id, name, plan, created_at	The customer organisation. Root of all isolation.
users	id, tenant_id, email, role	Responders / admins who manage the page.
status_pages	id, tenant_id, slug, custom_domain, branding, is_public	A tenant's page (a tenant may have more than one).
component_groups	id, status_page_id, name, position	Visual grouping of components.
components	id, status_page_id, group_id, name, current_status, position	A service whose health is shown.
incidents	id, status_page_id, title, status, impact, source, state, started_at, resolved_at	The incident record. state = proposed / published; source = manual / auto.
incident_updates	id, incident_id, body, status, author_id, is_proposed, created_at	Threaded updates posted during an incident.
incident_components	incident_id, component_id, impact_status	Join: which components an incident hits, and how hard.

TABLE	KEY COLUMNS	PURPOSE
<code>maintenances</code>	<code>id</code> , <code>status_page_id</code> , <code>title</code> , <code>status</code> , <code>scheduled_start</code> , <code>scheduled_end</code>	Planned maintenance windows.

Communication & ingestion tables

TABLE	KEY COLUMNS	PURPOSE
<code>subscribers</code>	<code>id</code> , <code>status_page_id</code> , <code>channel</code> , <code>address</code> , <code>verified_at</code>	Someone who opted in to updates.
<code>subscriptions</code>	<code>id</code> , <code>subscriber_id</code> , <code>scope</code> , <code>component_id</code>	Whole-page or per-component subscription.
<code>alert_sources</code>	<code>id</code> , <code>tenant_id</code> , <code>name</code> , <code>type</code> , <code>token_hash</code> , <code>rate_limit</code>	A configured signal source with its webhook token.
<code>alerts</code>	<code>id</code> , <code>alert_source_id</code> , <code>dedup_key</code> , <code>status</code> , <code>payload</code> , <code>received_at</code> , <code>incident_id</code>	Raw signals received; linked to an incident if they spawn one.
<code>templates</code>	<code>id</code> , <code>tenant_id</code> , <code>name</code> , <code>body_template</code> , <code>applies_to_status</code>	Pre-approved message templates for the approval mode.
<code>notifications</code>	<code>id</code> , <code>incident_update_id</code> , <code>subscriber_id</code> , <code>channel</code> , <code>status</code> , <code>idempotency_key</code> , <code>attempts</code>	One delivery attempt per subscriber per update. The idempotency key prevents duplicates.
<code>snapshots</code>	<code>id</code> , <code>status_page_id</code> , <code>version</code> , <code>storage_key</code> , <code>published_at</code>	Published page versions written to the snapshot store.
<code>audit_log</code>	<code>id</code> , <code>tenant_id</code> , <code>actor</code> , <code>action</code> , <code>target</code> , <code>created_at</code>	Who did what; every publish and approval recorded.

TENANCY NOTE

We start with a shared schema and a `tenant_id` column on every table, which is simplest to operate. If isolation needs grow, the same model extends to row-

level security or schema-per-tenant without redesigning the entities.

07

State diagrams

The lifecycles that govern incidents, component health, and maintenance.

7.1 Incident lifecycle

An incident moves `investigating` → `identified` → `monitoring` → `resolved`, with an optional `postmortem` afterwards. Auto-detected incidents enter through a `proposed` gate before becoming public.



DIAGRAM PLACEHOLDER · 06

Incident lifecycle state machine

States and transitions, including the `proposed` → `published` approval gate.

Eraser flowchart script to follow.

7.2 Component health

A component sits in one of: `operational` `degraded` `partial outage` `major outage` `maintenance`. Health is set by an active incident affecting it, by maintenance, or manually.



DIAGRAM PLACEHOLDER · 07

Component health state diagram

Transitions between health states and what triggers each. *Script to follow.*

7.3 Maintenance lifecycle

Maintenance runs `scheduled` → `in_progress` → `completed` , flipping affected components into the maintenance state for the window.



DIAGRAM PLACEHOLDER · 08

Maintenance lifecycle state diagram

Three-state lifecycle with the component-state side effects. *Script to follow.*

08

API surface

Four groups of endpoints, split by who calls them.

Public read API read plane

Served via the CDN as cached JSON. No auth; safe to hammer.

METHOD	PATH	RETURNS
GET	<code>/v1/pages/{slug}/status</code>	Current components, states, active incidents.

METHOD	PATH	RETURNS
GET	<code>/v1/pages/{slug}/incidents</code>	Past-incident history.
GET	<code>/v1/pages/{slug}/feed.rss</code>	RSS feed of updates.

Admin / write API control plane

Authenticated, tenant-scoped. Drives the dashboard.

METHOD	PATH	ACTION
POST	<code>/v1/incidents</code>	Create an incident.
POST	<code>/v1/incidents/{id}/updates</code>	Post an update (manual / template / approve).
PATCH	<code>/v1/components/{id}</code>	Set component health.
POST	<code>/v1/maintenances</code>	Schedule maintenance.
POST	<code>/v1/incidents/{id}/approve</code>	Approve a proposed update.

Subscriber API

METHOD	PATH	ACTION
POST	<code>/v1/pages/{slug}/subscribers</code>	Subscribe (email / SMS / webhook).
DELETE	<code>/v1/subscribers/{token}</code>	Unsubscribe.

Ingestion API async in

METHOD	PATH	ACTION
POST	<code>/v1/ingest/{source_id}?token=...</code>	Receive an alert event (with dedup key) from a signal source.

09

Deployment & monitoring

How it runs, how we watch it, and what it costs.

The read plane is just object storage behind a CDN, with nothing to "run." The control plane is a containerised backend plus its database, cache, and queue. Early on this fits a simple managed platform; at scale it moves to managed container orchestration. The notification workers scale independently of the API, since fan-out load is bursty and separate from request load.



DIAGRAM PLACEHOLDER · 09

Deployment topology

CDN + object storage on the read side; containerised API, database, cache, queue, and workers on the control side. *Script to follow.*

Monitoring our own system

Health endpoints on the API (liveness / readiness) so the platform can restart or drain unhealthy instances.

Publish lag. Time from update to live snapshot. The key product SLO.

Notification delivery. Queue depth, success/failure rates, retries.

Ingestion. Dropped-by-dedup vs accepted, per source.

COST POSTURE

This design is cost-effective by shape. Reads, which are the overwhelming majority of traffic, are static files on a CDN, costing pennies even during a 100K-RPS spike, and never touch the database. Writes are rare, so the database and API stay small. Workers and the queue cost almost nothing at

this write volume, and using third-party email means no mail infrastructure to run. (Detailed dollar modelling is future work.)

10

Trade-offs & future work

What we chose to defer, and why that's safe.

ITEM	STATUS	NOTE
Surviving our own outage (active failover)	deferred	The read/write split already makes the page outage-tolerant; full multi-region failover machinery comes later.
Real message queue + worker fleet	extension point	Seam designed in now; the real queue is the first scale-up lever.
Migrating off other vendors	out of scope	Clean extension point, not designed here.
Deeper AI automation	future	Drafting and postmortems first; broader automation stays human-approved.
Infra-as-code & metrics dashboards	extension point	Design accommodates Terraform / Prometheus-style tooling as the system grows.

What comes next: the LLD phase

This document is the high-level design. The low-level design follows separately: class diagrams (the publisher, the dispatcher, the ingestion adapters), sequence diagrams (a full incident from signal to notification), and folder structure following clean-architecture layering. Those Eraser scripts will slot into a dedicated LLD document.

Massive cacheable reads on one side, rare expensive writes on the other, a single publish gate between them. Keep that split intact and the rest of the system stays simple, cheap, and resilient.

Status Page Platform · System Design · v0.1

High-level design · diagrams pending in Eraser · low-level design to follow